Modular Verification for Earliest Deadline First Scheduler Implementation

Anonymous Authors # 82

***Abstract*—Real-Time Operating Systems (RTOSes) rely on scheduling algorithms to ensure predictable and deterministic task execution. The correctness of scheduler implementations is essential, as any deviation from specified scheduling behavior can lead to unintended and potentially catastrophic outcomes in safety-critical applications. While recent efforts have focused on developing formally verified schedulers or even entire RTOSes from scratch, widely used open-source RTOS implementations also require formal verification to ensure reliability. This need is further amplified by the fact that some EDF scheduler implementations reuse infrastructure originally designed for fixed-priority scheduling, increasing the risk of subtle semantic mismatches.**

In this work, we identify two essential properties that any implementation of the Earliest Deadline First (EDF) scheduler must satisfy. Based on these properties, we propose a systematic verification framework utilizing deductive verification, that applies to any EDF-based scheduler realization. To demonstrate the applicability of our framework, we conduct a case study on the EDF scheduler implementation in the widely used open-source RTOS, i.e., RTEMS 5. Our verification confirms the correctness of the EDF implementation under specific assumptions, offering assurance for real-time systems employing RTEMS and demonstrating the practicality of the proposed verification framework.

***Index Terms*—EDF Scheduling, Formal Verification, Real-Time Systems, Real-Time Operating Systems**

I. Introduction

Real-time systems play an indispensable role in modern computing, delivering timely and predictable responses is crucial across various domains, such as aerospace, automotive, medical devices, and industrial control. In these contexts, Real-Time Operating Systems (RTOSes) act as the backbone, managing hardware resources and ensuring that real-time tasks are executed within their specified deadlines. RTOSes distinguish themselves from general-purpose operating systems through their ability to provide deterministic and timely responses.

Ensuring the correctness of an RTOS relies on both the theoretical design and precise implementation. While mathematical verification of the RTOS design provides a robust foundation, it is equally imperative to validate that the implementation adheres closely to this proven design. Even minor discrepancies between a verified design and its implementation can introduce vulnerabilities, particularly in safety-critical systems.

Guaranteeing that the implementation of an RTOS is fully realized and performs reliably under the conditions for which it is designed typically involves two approaches: a) Developing a formally verified RTOS from scratch, building on a solid theoretical foundation, such as the seL4 microkernel [26] and CertiKOS [21]. b) Establishing formal processes to verify the functionality of existing RTOSes. This work focuses on the latter approach: the formal verification of an existing RTOS. This is particularly significant due to the prevalence of RTOSes developed through community efforts, such as FreeRTOS [3], Zephyr [14] and RTEMS [34], which have been widely used, including in safety-critical scenarios. However, comprehensive verification in practice faces significant challenges, which arise from both the limitations of current verification tools and the legacy features of the RTOS's APIs and internal structures. To tackle these challenges, a modular-based verification strategy is commonly employed. This method involves verifying individual components ~~of the RTOS~~ in isolation, based on the assumption that other components within the ~~system~~ RTOS are correctly implemented. *Model Checking* [5], [18] and *Deductive Verification* [28], [32], [33] are commonly utilized methodologies for the formal verification of module correctness within a specified RTOS.

~~In any RTOS, the~~ The scheduler is one of the most critical components, responsible for managing the execution of real-time tasks to meet their timing requirements. Consequently, formally verifying the scheduler's functionality is essential for ensuring the overall reliability and efficiency of an RTOS, as it guarantees adherence to task priorities and deadlines. In practice, fixed-priority scheduling is one of the most widely adopted algorithms across modern RTOSes due to its simplicity and dependable performance, such as the fixed-priority preemptive scheduling in FreeRTOS, ~~static priority~~ static-priority scheduling in Zephyr, and deterministic priority scheduling in RTEMS.

The Earliest Deadline First (EDF) scheduling policy, on the other hand, has been extensively studied for its theoretical optimality in preemptive scheduling [9]. However, EDF ~~'s reliance on a dynamic priority~~ relies on a dynamic-priority system, where ~~tasks~~ jobs with earlier deadlines are assigned higher priorities, introduces complexity. This approach requires frequent priority updates and incurs considerable overhead for queue management and scheduling decisions. As a result, few RTOSes support EDF scheduling, e.g., RTEMS and Zephyr, which still rely on fixed-priority primitives as a foundation for EDF implementations.

In practice, to enhance performance and minimize overhead, certain optimizations are often applied in the implementation, which may necessitate trade-offs between exact adherence to the design and practical efficiency. For example, Zephyr employs the `_kernel.ready_q.cache` to store the next job to execute, reducing the overhead associated with searching for the next job in the ready queue. However, this optimization can lead to unintended task execution orders under certain conditions. As reported in [2], issues may arise if the ready

queue cache is not updated correctly. Specifically, due to this design choice, each job must update its absolute deadline before being added to the ready queue, as no further deadline updates are permitted once the job is in the queue unless the `_kernel.ready_q.cache` is explicitly refreshed. While thorough documentation can help avoid misuse or misinterpretation of such mechanisms, it alone cannot guarantee correctness. Additionally, some components or functionalities may be added at later stages, relying on and reusing original primitives not specifically designed for them to maintain compatibility with the existing RTOS structure. This gap between theoretical models and practical considerations highlights the critical need for a comprehensive verification.

To the best of our knowledge, only a few of the existing work consider formal verification of a scheduler. For example, Vanhems et al. [35] proposed a formal proof methodology for EDF scheduling by reimplementing the scheduler atop an existing system, limiting its relevance to legacy RTOSes like FreeRTOS and RTEMS. Besides, the Correctness Verification Method (CVM) proposed by Liang et al. [30] verifies 22 API functions in the scheduling module of FreeRTOS6.1.1. However, it lacks a systematic framework for reasoning about the correct composition of these functions to enforce a specific scheduling policy, limiting its ability to detect emergent behaviors or subtle bugs from valid but unintended API sequences. ~~Recently,~~ Shi et al. [33] proposed a verification framework for synchronization protocols, assuming the scheduler is already correct, an assumption that may not hold in all RTOSes.

In this work, we propose a general framework ~~to ensure~~ for verifying the correctness of EDF scheduler implementations in RTOSes. ~~We identify two fundamental correctness properties, i.e., deadline-driven priority assignment and preemptive scheduling, and define a generic software architecture for EDF schedulers. We propose a deductive verification methodology grounded in modular analysis, and show how it can be applied across different RTOS implementations. The framework captures essential EDF components and typical integration patterns, including implementations that reuse existing infrastructure originally developed for fixed-priority scheduling.~~

~~To demonstrate the applicability of our framework, we conduct a case study on RTEMS, where the EDF scheduler is built atop the primitives of its Deterministic Priority Scheduler, originally intended for fixed-priority semantics. Despite this legacy structure, our verification confirms that the EDF scheduler implementation satisfies the required correctness properties under clearly defined assumptions. This case study highlights the practical relevance of our framework for verifying EDF schedulers embedded in realistic RTOS architectures.~~

Our Contribution: To the best of our knowledge, this is the first work to ~~propose~~ present a formal verification framework applicable to EDF scheduler implementations in existing RTOSes. The ~~main~~ key challenges lie in identifying ~~core~~ key correctness properties, formalizing them within a general framework, and addressing the reuse of generic kernel primitives ~~that may not~~ not originally designed for dynamic-priority scheduling. The contributions of this work are summarized as follows:

- We analyze the general operational workflow of EDF schedulers in RTOSes and formally define two key correctness properties, i.e., *deadline-driven priority assignment* and *preemptive scheduling behavior*, that any correct implementation must satisfy.
- We describe a generic software structure and implementation pattern for EDF schedulers, covering essential components such as task/job models, periodic release, ready queue management, and preemptive dispatching.
- We propose a modular ~~deductive verification framework and~~ verification framework for formally verifying EDF scheduler implementations against the two defined correctness properties under clearly stated assumptions. We demonstrate its applicability by verifying the compliance of a generic EDF implementation with ~~the defined~~ these properties.
- We validate our framework through a case study of the uniprocessor EDF scheduler in RTEMS ~~5.~~ 5, confirming that the implementation satisfies the specified correctness properties. All verification artifacts, including ~~ACSL~~ verification contracts and replication instructions, are made publicly available [4].

II. Background

In this section, we first introduce the task model assumed in this work. We then describe the fundamental principles of the EDF scheduling algorithm. Finally, we provide a formal definition of the key properties that any correct EDF scheduler implementation must satisfy.

A. Periodic Task Model

We consider the implemented scheduler is employed on a uni-processor system for a set of N tasks $\mathbb{T} = \{\tau_1, \ldots, \tau_N\}$. Each task is described by ~~$\tau_i = (C_i, D_i, T_i)$~~ $\tau_i = (D_i, T_i)$, where:

- ~~C_i denotes the worst case execution time (WCET) of τ_i.~~
- T_i is the period of task τ_i.
- D_i is the relative deadline of τ_i.

All tasks release an infinite number of task instances, called jobs. Each job is characterized by a tuple ~~$J_i^\ell = (\tau_i, r_i^\ell, C_i, d_i^\ell)$~~ $J_i^\ell = (\tau_i, r_i^\ell, d_i^\ell)$. For the ℓ-th job, $\ell \in \mathbb{N}$: r_i^ℓ is the release time, i.e., $r_i^\ell = \ell \times T_i$, and d_i^ℓ is the absolute deadline, i.e., $d_i^\ell = r_i^\ell + D_i$. Please note that the worst-case execution time (WCET) C_i of task τ_i is not relevant to the EDF scheduler's operation and is thus omitted from the system model.

Let the ready queue $\mathbf{Q}(t) = \{J_1, J_2, \ldots, J_n\}$ be a set of jobs sorted in non-decreasing order of their absolute deadlines. For any two jobs $J_i^i, J_j^j \in \mathbf{Q}(t)$ must satisfy the following condition: if J_i^i precedes J_j^j in $\mathbf{Q}(t)$, then $d_i^i \leq d_j^j$. If $d_i^i = d_j^j$, then J_i^i must be released earlier than J_j^j. In addition, the

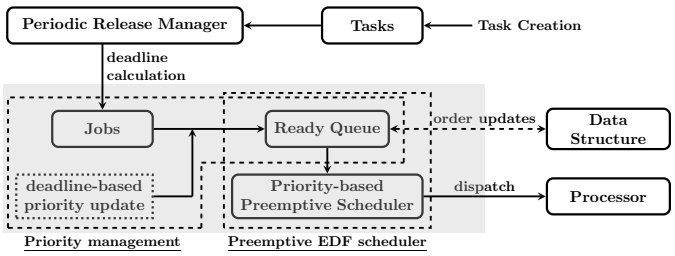

Fig. 1. The workflow of EDF scheduler. The gray area is the verification target in this work.

schedule function $\mathbf{S} : \mathbb{N} \to J \cup \{\perp\}$ maps each time t to the job J executing at time t, or $\perp$ if the processor is idle.

B. EDF Scheduling Workflow and Core Components

Preemptive EDF scheduling guarantees that, at any moment, the job with the earliest absolute deadline among all ready jobs is selected for execution. Figure 1 illustrates the general operational workflow of an EDF scheduler, abstracted across RTOS implementations. The process begins with task creation. The Periodic Release Manager handles task activations by generating jobs at each release point, computing absolute deadlines, and assigning priorities accordingly. Afterwards, released jobs are inserted into the ready queue with deadline-based ordering. The Priority-Based Preemptive Scheduler monitors the queue and the currently executing job, performing preemptions when a new job arrives with an earlier deadline.

While the core scheduling principle is well-defined, its realization in RTOS kernels can vary significantly due to legacy design choices, internal API constraints, and system-level optimizations. Despite these differences, most EDF implementations share two core functionalities, including:

- Priority management: EDF is a job-level dynamic-priority scheduler that assigns priorities to ~~tasks~~ jobs based on their absolute deadlines, i.e., the earlier the deadline, the higher the priority. Since EDF dynamically adjusts priorities, they must be recalculated whenever a new job arrives or an existing job's deadline is modified. This process involves updating the job's priority and repositioning it within the ready queue to maintain correct order. ~~or a red-black tree.~~
- Preemptive EDF scheduler: Once priorities are updated, due to job arrivals, completions, or deadline changes, the scheduler always selects the job with the earliest deadline, i.e., the head of the ready queue, for execution. EDF operates in a preemptive manner. That is, if a new job arrives with an earlier deadline than the currently running job, it immediately preempts the executing job on the processor. This triggers a context switch.

When multiple jobs in the ready queue share the same priority (i.e., identical absolute deadlines), the scheduling order is implementation-defined. For example, in the RTEMS implementation, tie-breaking is resolved using FIFO semantics, i.e., the job enqueued earlier is given the higher priority.

C. Formal Properties Required for EDF Scheduler

To ensure correctness in practical RTOS implementations, the foundational principles of EDF scheduling must be preserved despite system-specific design constraints. Based on the preceding analysis, we distill the following essential properties that any correct EDF implementation must satisfy:

Property 1: Deadline-Driven Task Prioritization This property asserts that, among all jobs in the ready queue, the job with the earliest absolute deadline is assigned the highest priority. Formally, $\forall J_i^i, J_j^j \in \mathbf{Q}(t)$, job J_i^i has a higher priority than job J_j^j, if $d_i^i < d_j^j$, and ties are broken arbitrarily. We assume that the $Q(t)$ does not include the current executing job. Specifically, when the processor is idle, the scheduler must take a job from the non-empty ready queue, i.e., $J_{sched} = \arg\min_{J_i^i \in \mathbf{Q}(t)} d_i^i$, and the ready queue is updated as: $\cancel{\mathbf{Q'}(t) = \mathbf{Q}(t) \setminus J_{sched}}\mathbf{Q}(t) = \mathbf{Q}(t) \setminus J_{sched}$. The scheduler's behavior is defined by the rule: $\mathbf{S}(t) = J_{sched}$.

Property 2: Preemptive EDF Scheduling In preemptive EDF scheduling, the currently executing job ~~will be~~ is interrupted and replaced if a newly released job has an earlier absolute deadline. Specifically, at any time t, if a new job J_j^j arrives with $d_j^j < d_i^i$ where $\mathbf{S}(t) = J_i^i$, then: $\mathbf{S}(t) = J_j^j$ and J_i^i is preempted. The preempted job J_i^i will be pushed back to the ready queue $\cancel{\mathbf{Q'}(t) = \mathbf{Q}(t) \cup J_i^i}\mathbf{Q}(t) = \mathbf{Q}(t) \cup J_i^i$.

III. General EDF Scheduler Design and Implementation

Building on the previously discussed workflow and core components, this section abstracts the general software structure of an EDF scheduler. We then present representative implementation patterns for each component, highlighting common design choices and integration strategies.

A. General Software Structure for EDF Scheduling

This subsection outlines the key components required for EDF scheduling: the task model, periodic release manager, jobs, ready queue, and priority-based preemptive scheduler.

1) Tasks: A task represents a long-lived software entity in an RTOS, typically defined at system configuration time. Tasks encapsulate application-level control flows and may be periodic, sporadic, or aperiodic. In EDF scheduling, tasks themselves do not carry static priorities; rather, they act as templates for generating jobs, which are assigned dynamic priorities based on their absolute deadlines.

Each task is associated with a set of configuration parameters that determine its timing behavior, i.e., $\cancel{\tau_i = (C_i, D_i, T_i)}\tau_i = (D_i, T_i)$. These configurations are typically specified during system initialization but may be updated during runtime to accommodate dynamic system behavior. Each task is represented internally using a Task Control Block (TCB), which stores configuration data and runtime information such as next release time and pointers to pending jobs.

2) Periodic Release Manager: The Periodic Release Manager (PR Manager) is responsible for generating jobs according to the temporal behavior specified by each task, including periodic, sporadic, or event-driven releases. It enforces the timing constraints of tasks and ensures that job creation aligns with the system's real-time requirements.

For each periodic task, the PR Manager maintains a release schedule, typically driven by a system timer. When the system clock reaches a task's next release time, the PR Manager triggers a new job release and:

- Allocates a new job structure from its pending job queue;
- Computes the absolute deadline as the sum of the current time and the task's relative deadline;
- Assigns the job's priority equal to its absolute deadline;
- Inserts the job into the ready queue for scheduling.

~~In practical implementations, the PR Manager may be integrated with the system's timer interrupt handlers or event scheduling subsystem.~~

3) Jobs: A job is a runtime instantiation of a task, characterized by a specific arrival time and absolute deadline. Each job transitions through the following life cycle states: *Executing*, *Ready*, *Blocked*, *Dormant*, and *Non-existent*. Most state transitions are triggered by scheduling decisions. For example, a job may transition to the Blocked state after completing execution before its deadline, or from Executing to Ready if preempted by a higher-priority job.

In EDF scheduling, a job's priority is ~~dynamically derived from~~ defined by its absolute deadline, with earlier deadlines assigned higher priority (i.e., lower numerical values in typical RTOS encodings). Each job maintains key attributes: arrival time, absolute deadline, ~~remaining execution time,~~ dynamic priority, and a pointer to its associated task. The job structure also includes hooks for priority updates and links for insertion into the ready queue.

4) Ready Queue: The ready queue holds all jobs that are ready to execute but not currently running. In an EDF scheduler, the queue is maintained in strict order of job priorities, determined by absolute deadlines. Efficient management of the queue is essential to uphold EDF semantics, typically implemented using red-black trees or binary heaps to support fast insertion, removal, and priority updates.

The ready queue provides the following core operations:

- Inserting a newly released or preempted job with its computed priority;
- Updating a job's position when its deadline changes;
- Retrieving the job with the earliest deadline (the head of the queue) for scheduling decisions.

The correctness and efficiency of the ready queue are critical to ensuring timely and predictable scheduling behavior.

5) Priority-Based Preemptive Scheduler: The priority-based preemptive scheduler selects and dispatches the job with the highest priority (i.e., earliest deadline) for execution. It continuously monitors job arrivals, completions, and priority changes to determine whether a context switch is required.

Key operations include:

- Comparing the priority of a newly released or updated job with that of the currently running job;
- Initiating preemption when a new job has a higher priority, triggering a context switch;
- Dispatching the job with the earliest deadline from the ready queue when the processor is idle or when the current job completes;
- Performing context switches: saving the state of the current job, enqueuing it back into the ready queue, and restoring the context of the next job.

The scheduler ensures that the job with the earliest deadline always gains processor access, faithfully enforcing EDF scheduling semantics.

B. Exemplary Implementation of the EDF Scheduler

This subsection presents an exemplary RTOS kernel implementation of the EDF scheduler, structured in two parts: 1) priority management and ready queue handling, covering job creation through insertion into the ready queue, and 2) the priority-based preemptive scheduler, managing job dispatch and context switching.

1) Job Release Workflow and Ready Queue Management: Listing 1 illustrates a general kernel-level implementation of EDF scheduling components, covering job release, priority assignment, and ready queue operations.

The `edf_assign_priority` function computes a job's absolute deadline and assigns its EDF priority accordingly. The `edf_compare_jobs` function defines the priority ordering for the ready queue, ensuring that jobs with earlier deadlines have higher scheduling priority.

The ready queue is abstracted via `ready_queue_t`, with placeholder functions for insertion, removal, and retrieval. The `release_job` function demonstrates a typical workflow, i.e., assigning priority to a newly released job, inserting it into the ready queue, and updating the task's dynamic state, e.g., active job count and next release time.

In practice, the PR Manager may be integrated with the system's timer interrupt handlers or event scheduling subsystem. Sporadic job releases can be supported by replacing or extending the job release mechanism accordingly. This implementation reflects common RTOS practices, where EDF priorities are derived directly from absolute deadlines~~, and ready queue management is optimized for fast retrieval of the highest-priority job.~~.

2) EDF Dispatch and Preemption Logic: The implementation of the core functionality of a priority-driven (deadline-based) preemptive scheduler is shown in Listing 2. This example demonstrates how the scheduler selects and dispatches the highest-priority job, handling both normal dispatch and preemption cases based on EDF semantics.

IV. Formal Verification Framework for EDF Scheduler

This section presents a modular verification framework ~~for EDF schedulers, targeting core scheduling components under the assumption that lower-level RTOS primitivesand~~

```c
#define MAX_PRIORITY_VALUE 0xFFFFFFFF
typedef uint32_t time_t;

// Priority Assignment Logic
void edf_assign_priority(job_t *job, time_t current_time) {
  job->release_time = current_time;
  job->absolute_deadline = current_time + job->relative_deadline;
  job->priority = job->absolute_deadline;
}

// Comparison Function for Ready Queue
int edf_compare_jobs(const job_t *a, const job_t *b) {
  // Lower deadline means higher priority
  if (a->priority < b->priority) return -1;
  if (a->priority > b->priority) return 1;
  return 0;
}

// Simplified ready queue (tree, heap, etc.)
typedef struct ready_queue_t {
  job_t *head; // Pointer to the earliest deadline job
} ready_queue_t;

// new job insertion
void ready_queue_insert(ready_queue_t *queue, job_t *job) {
  if (queue->head == NULL || edf_compare_jobs(job, queue->head) < 0) {
    job->next_in_queue = queue->head;
    queue->head = job;
  } else {
    job_t *current = queue->head;
    while (current->next_in_queue != NULL &&
    edf_compare_jobs(job, current->next_in_queue) >= 0) {
      current = current->next_in_queue;
    }
    job->next_in_queue = current->next_in_queue;
    current->next_in_queue = job;
  }
}

// job removal
void ready_queue_remove(struct ready_queue_t *queue, struct job_t *job);

// return the job with highest priority
job_t* ready_queue_peek(ready_queue_t *queue) {
  return queue->head;
}
// Kernel Usage Example
void release_job(job_t *job, time_t current_time) {
  edf_assign_priority(job, current_time);
  ready_queue_insert(job);
  task->active_job_count++;
  task->next_release_time = current_time + task->period;
}
```

Listing 1. Illustrative example of job release and readyqueue management.

~~ready queue mechanisms are correctly implemented. We apply deductive verification using Frama-C and ACSL to establish compliance with EDF semantics~~targeting the core components of EDF scheduler implementations in RTOSes. In contrast to the approach of [35], which verifies a reimplemented EDF scheduler built on newly designed primitives, our goal is to formally verify existing implementations embedded within RTOS kernels, potentially relying on generic infrastructure not originally intended for EDF scheduling.

```c
void scheduler_tick(struct ready_queue_t *queue, struct job_t *new_job) {
  if (current_job == NULL) {
    // Idle CPU: dispatch highest-priority job
    current_job = ready_queue_peek(queue);
    if (current_job != NULL) {
      dispatch(current_job);
    }
  } else if (new_job != NULL && new_job->priority < current_job->priority) {
    // Preemption case
    save_context(current_job);
    ready_queue_insert(queue, current_job);
    current_job = new_job;
    ready_queue_remove(queue, new_job);
    dispatch(current_job);
  }
}
```

Listing 2. The illustrative example of preemptive scheduler.

A. Deductive Verification

In order to verify the properties of the EDF scheduler implementation, we apply deductive verification [13], [24]. For completeness, we briefly review Hoare Logic and the concept of the weakest precondition.

Hoare Logic provides a systematic framework for rigorously reasoning about program correctness [24]. A key concept is the *Hoare Triples*, denoted as $\{P\}\ C\ \{Q\}$, where:

- C is the set of considered instructions in the program.
- P is the *precondition*, describing the state before executing C.
- Q is the *postcondition*, describing the expected state after C's execution.

If the precondition P holds before execution, and if C terminates, the postcondition Q must hold afterward. The weakest precondition, denoted as $wp(C, Q)$, represents the minimal condition that guarantees Q after executing C.

To express these properties, we use the ANSI/ISO C Specification Language (ACSL) [16], which enables the formal specification of function contracts directly in C code. Contracts are embedded using comments (`/*@ ... */`) and include:

- `requires` for preconditions;
- `assigns` to declare which variables may be modified;
- `ensures` for postconditions;
- `\valid` to specify valid memory regions.

ACSL also supports *ghost variables*, *ghost pointers*, and *ghost code*, used to assist verification tools without affecting runtime behavior. These ghost elements enable reasoning about internal states, historical data, or complex data structures for verification, enhancing the precision of formal proofs while keeping the functional code unchanged.

B. Frama-C and Memory Model

To validate the function contracts, we used Frama-C (Framework for Modular Analysis of C programs) version 25 [12], [15], along with the WP plugin [17] for static analysis and deductive verification. The WP plugin generates proof obligations, i.e., logical formulas derived from the code and its annotations, which are initially simplified using the built-in

Qed engine. If Qed cannot fully resolve them, the obligations are passed to an SMT solver via Why3 [7]. In this work, we used Alt-Ergo version 2.4.2 [1] as the backend solver.

To ensure compatibility with codebases that rely on GCC-specific extensions, Frama-C's machdep settings provide support for features such as empty structs and flexible array members. Verification is performed at the source-code level, making the approach architecture-independent and applicable across different hardware platforms, assuming a conforming implementation of lower-level system components.

Accurate verification of C code requires a formal memory model to represent read and write operations, particularly when accessing memory via pointers. We applied the *Typed* memory model [19], which represents memory using global arrays and models pointers as indices. The `\separated` predicate ensures that distinct pointers refer to disjoint memory regions, preventing aliasing issues and unintended interactions between components. Besides, the `Cast` option was enabled to support unsafe pointer casts, which are often used in system-level code to implement low-level abstractions and structural inheritance.

C. Key Assumptions

Our verification framework assumes that the EDF scheduler is implemented on a correctly implemented RTOS infrastructure. This includes the correctness of all underlying kernel primitives, such as context switching, dispatching mechanisms, and memory management, as well as the data structures used in the scheduler's implementation.

Specifically, we assume the correctness of the ready queue implementation and its core operations, e.g., insertion, deletion, and retrieval. In systems like RTEMS, the ready queue is typically implemented using the OpenBSD red-black tree library. Due to its macro-intensive structure and low-level design, direct ACSL-based annotation and verification are impractical. However, its widespread adoption, thorough documentation, and long-term use in production systems support its trustworthiness.

Accordingly, we treat the red-black tree as a trusted component and assume the correctness of its structural and query operations. This allows us to focus verification efforts on EDF-specific logic while treating the priority queue as a reliable abstraction. For stronger formal guarantees, verified alternatives such as the priority search trees in [27] could be adopted in future work.

D. Verification of EDF Scheduler Functionalities

This subsection presents the formal verification of the EDF scheduler's core components using Frama-C. We divide the verification into two parts: 1) priority management, including job release, priority assignment, and ready queue insertion; and 2) preemptive scheduling and dispatch.

1) Verification of Priority Management: We aim to verify that when a job is released: 1) its priority is correctly assigned based on its absolute deadline, and 2) it is correctly inserted into the ready queue. The detailed ACSL contracts are shown

```
/*@
  requires job->relative_deadline <= MAX_PRIORITY_VALUE
   - current_time;
  assigns job->release_time, job->absolute_deadline,
  job->priority;
  ensures job->release_time == current_time;
  ensures job->absolute_deadline == current_time + job
  ->relative_deadline;
  ensures job->priority == job->absolute_deadline;
*/
void edf_assign_priority(job_t *job, time_t current_time)

/*@
  requires \valid(queue);
  requires \valid(job);
  assigns queue->head;
  ensures queue->head != \null;
  ensures queue->head->priority <= job->priority;
*/
void ready_queue_insert(ready_queue_t *queue, job_t *job)
```

Listing 3. The illustrative example verification for priority management.

in Listing 3. The `edf_assign_priority()` function ensures that the job's priority reflects its absolute deadline. The `ready_queue_insert()` contract verifies that the ready queue is correctly updated after insertion, ensuring that the head of the queue is always the job with the earliest deadline.

~~Please note that the internal~~ As discussed in Section IV-C, we assume the correctness of the ready queue implementation~~is not formally verified in this work, as its underlying data structure is abstracted and not specified. We assume the ready queue operates correctly, maintaining proper ordering and integrity as required by EDF semantics.~~. Therefore, internal data structure correctness is not subject to formal verification here.

2) Verification of Preemptive Scheduling and Dispatch: Here we verify two key properties of the scheduler: 1) it always dispatches the job with the earliest deadline, and 2) preemption occurs correctly when a higher-priority job arrives.

The detailed ACSL contracts are provided in Listing 4. The verification ensures that the scheduler upholds EDF semantics by maintaining correct dispatch and preemption behavior. Specifically, the ACSL contract guarantees that after execution, either no job is running or the currently running job has the highest priority among all jobs in the ready queue. This enforces the core EDF property that lower-priority jobs cannot preempt higher-priority ones and validates the correctness of the context-switching mechanism, ensuring the system always runs the job with the earliest absolute deadline.

V. EDF Implementation in RTEMS

To validate the proposed framework, we conduct a case study on the EDF scheduler implementation in ~~RTEMS5.~~ RTEMS 5. This section details the realization of each core component, following the workflow outlined in Figure1.

In RTEMS, threads serve as the practical representation of jobs for scheduling related operations. Therefore, "thread" and "job" are employed interchangeably in different contexts:

```
/*@
  requires \valid(queue);
  requires \valid(new_job);
  assigns current_job;

  behavior no_preemption:
  assumes current_job == \null || new_job->priority >= current_job->priority;
  ensures current_job == \old(current_job);

  behavior preemption:
  assumes current_job != \null && new_job->priority < current_job->priority;
  ensures current_job == new_job;
  ensures in_ready_queue(queue, \old(current_job));

  ensures current_job == \null || (\forall job_t *j; in_ready_queue(queue, j) ==> current_job->priority <= j->priority);
*/
void scheduler_tick(ready_queue_t *queue, job_t *new_job)
```

Listing 4. The illustrative example verification for priority management.

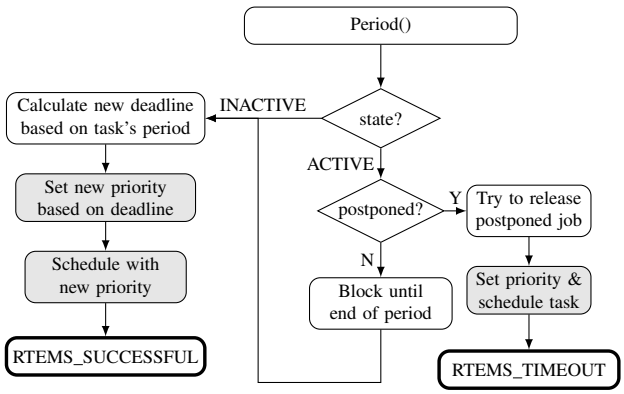

Fig. 2. The workflow of `period` directive. The highlighted blocks in gray are implemented in the scheduler and verified in this work.

"job" refers to the abstract concept in scheduling theory, while "thread" is used when discussing concrete implementations.

1) Periodic Release Manager: The `Rate Monotonic Manager`, together with the watchdog service are applied to handle the arrival of a job. A watchdog is triggered, by the `period` directive, and various functions are executed to operate the new release of the job. Note that *rate monotonic* is just a prefix for legacy APIs. In fact, these functions, as the name suggested, are originally used by the fixed-priority scheduling implementations in RTEMS. For the sake of clarity, however, we rename it as `PR manager` ~~in~~ for the rest of work.

Figure 2 shows the workflow of periodic job releases. The blocks in gray are implemented in the scheduler and verified as part of the EDF scheduler in this work. Based on the current state of the `period` directive, different operations are triggered accordingly, as follows:

- `INACTIVE`: When the first job of a task arrives, there are no jobs currently executing on the processor from the same task. A new deadline is therefore calculated and set to the watchdog, with the period length specified. Based on that deadline, a priority is set to this job and triggers possible context-switches on the current processor. The period status is set to `ACTIVE`.
- `ACTIVE`: When the job finishes its execution, it checks if any jobs were postponed due to missed deadlines:
  - No postponed jobs: the task blocks itself and wait for a new period to release a new job with a new priority.
  - At least one postponed job: The task releases the job based on the current assigned period priority and do not renew the watchdog timer. If the job has missed its deadline, then the function call results in an `RTEMS_TIMEOUT` return code. In that case the deadline miss has to be handled separately [10].

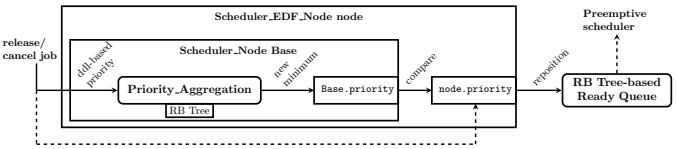

Fig. 3. The ready queue management workflow within EDF scheduler node in RTEMS implementation.

a) Job Priority Assignment: The ~~implicit deadline is implemented in RTEMS~~ implicit deadline is implemented by default, i.e., $\forall \tau_i,\ D_i = T_i$. Therefore, the priority assignment is managed by the PR manager. That is, when a new job arrives, its assigned priority is calculated, based on its absolute deadlines via the watchdog service.

b) Scheduler Node: In RTEMS, each processor must be explicitly assigned a scheduler instance during system initialization. This assignment is typically performed via configuration in a dedicated header file. At the core of RTEMS scheduling infrastructure lies the base scheduler node, which serves as a generic container for job-specific metadata required to make scheduling decisions. These decisions are commonly triggered by events such as job completions or changes in the ready queue. For each scheduling algorithm, the base scheduler node is extended to form an algorithm-specific scheduler node that includes additional data structures tailored to that algorithm's requirements. In the case of the EDF scheduler, this node is augmented with a red-black tree control node and a control priority field, both of which facilitate efficient priority management and ready queue organization using a red-black tree structure. The structural relationship is illustrated in Figure 3.

2) Ready Queue Management: Managing the ready queue is a complex process that intertwines with scheduling decisions. In the following, we introduce the whole workflow of how the ready queue is updated from a job event such as job release or cancel. The detailed workflow is shown in Fig. 3. We start from the key part, i.e., *priority aggregation* for each individual job, and followed by the updates in multiple priority fields of jobs for making scheduling decisions.

a) Priority Aggregation: In RTEMS, a job may be associated with multiple priority values arising from various

system mechanisms. The role of priority aggregation is to determine the job's effective priority by selecting the highest-priority value among all assigned priorities, i.e., the minimum numerical priority value. Please note, in RTEMS lower numbers indicate higher priority. Such multiplicity can stem from mechanisms like priority ceiling protocols, which may temporarily elevate a job's priority by registering additional ceiling priority nodes, or from multirate execution scenarios, where periodic job releases with varying deadlines result in changes to priority assignments over time.

Internally, RTEMS manages a red-black tree within the `Scheduler_Node Base` structure (specifically in `Priority_Aggregation` structure) to store all active priority nodes associated with a job. When a priority node is added, modified, or removed, the priority aggregation procedure is invoked to recompute the job's effective priority based on the updated set of priority nodes. In the typical EDF case where a job's priority is solely determined by its absolute deadline, only a single priority node exists in the tree. In such cases, priority aggregation simply returns the most recent priority value, which may change dynamically upon a new job release or a deadline update.

b) Updates in Multiple Priority Fields: Due to the internal architecture and legacy APIs in RTEMS, several distinct priority fields are used to implement the EDF scheduling mechanism. Figure 3 illustrates the relationships among these fields. At initialization, the `Scheduler_Node Base` and the `Scheduler_EDF_Node node` share the same priority value. Upon job release or cancellation, the job's priority is computed by the PR Manager based on its absolute deadline. This deadline-based priority is then inserted or updated in the red-black tree maintained by the `Priority_Aggregation` structure. If the new priority is smaller than the current minimum, the structure updates the job's effective priority, stored in `Base.priority` within the `Scheduler_Node Base`. Subsequently, this updated `Base.priority` is compared with the EDF-specific scheduler node priority stored in `node.priority`. If a mismatch is detected, `node.priority` is updated accordingly. This ensures that the job is correctly repositioned in the ready queue, allowing the EDF scheduler to make accurate and up-to-date scheduling decisions.

3) Context Switch Mechanism for Preemptive Scheduling: Preemptive scheduling is facilitated through a structured mechanism that ensures tasks with the earliest deadlines are prioritized for execution. After a task's priority is updated, the scheduler evaluates whether this task should preempt the currently running task. If the updated task has an earlier deadline, the scheduler designates it as the processor heir by updating the `_Thread_Heir` pointer. To signal the need for a context switch, the `_Thread_Dispatch_necessary` flag is set. The dispatcher then performs the common context switch operation.

VI. Verification of the EDF Scheduler in RTEMS

This section details the formal verification of the EDF scheduler implementation in RTEMS, focusing on the scheduler module and relevant `_Thread` functions. We outline the preprocessing steps, summarize key EDF-related functions, and explain their verification objectives. While full ACSL annotations are available in the repository [4], we emphasize functional behavior and verification rationale for clarity.

A. Setup and Source Code Processing

The applied version of Frama-C includes machdep support for GCC extensions, accommodating compiler-specific constructs used in RTEMS, such as empty structs and flexible array members. We employed the corresponding RTEMS cross-compiler, `x86_64-rtems5-gcc`, and selected the `AMD64` Board Support Package (BSP), the the BSP available for `x86_64` in RTEMS, supporting a uniprocessor configuration.

For the scheduler-specific code, annotating every function with a contract is unnecessary. Functions that do not affect the verification goals, such as `_Thread_Wait_acquire_critical`, which merely acquires a wait lock to guard shared resources, are excluded from verification using the `assigns \nothing` clause to explicitly indicate no effect on the state relevant to scheduling.

The EDF scheduler in RTEMS relies on the `tree.h` library from OpenBSD, which ~~implements~~ provides the red-black tree ~~crucial for~~ implementation used for ready queue management and priority aggregation within ~~the scheduler node. Verifying its correctness is particularly challenging due to the library's macro-heavy design and low-level operations, making it impractical for direct ACSL annotation. Therefore, we rely on its documentation and assume correctness for all critical operations, including insertion, deletion, search, rotations, and utility functions such as subtree replacement and minimum/maximum value retrieval. This assumption allows us to treat the dynamic priority queue operations for the EDF scheduler as correct. Alternatively, the red-black tree could be replaced by a verified implementation, such as the priority search trees in [27], providing formal guarantees for these core operations.~~

~~Although the red-black tree implementation itself is not formally verified~~scheduler nodes. As discussed in Section IV-C, we assume the correctness of this data structure. To facilitate modular verification of the scheduler logic, we annotate ~~its key functions to specify their expected functionality. Using~~ the tree's key interface functions and employ ghost pointers to abstract ~~results allows us to articulate the intended behavior of tree operations and maintain clarity within the verification framework~~return values, thereby preserving verification clarity without requiring internal tree validation.

B. Function Overview

This section outlines the EDF scheduler functions targeted for verification, including the thread-related functions invoked by the scheduler. The gray-shaded region in Figure 1 highlights the scope of the verification focus in this work. To verify

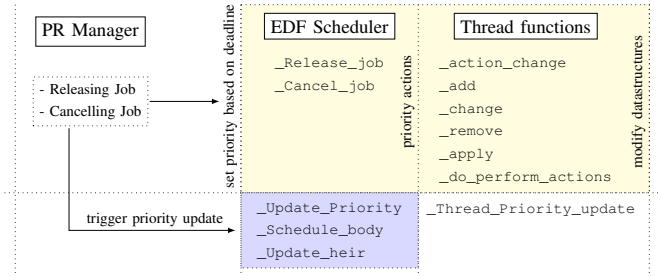

Fig. 4. Overview of verified EDF scheduler and thread priority functions, with common prefixes `_Scheduler_EDF` for EDF Scheduler related functions and `_Thread_Priority` for Thread related functions.

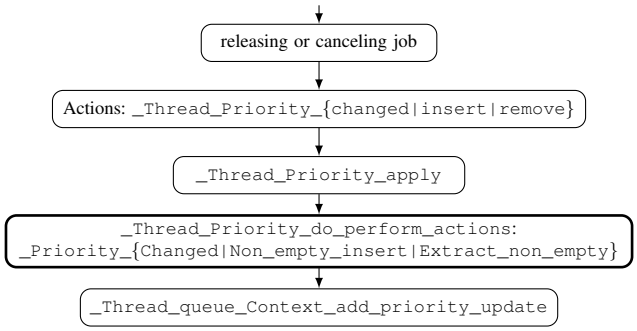

Fig. 5. Control flow of modifying the base scheduler nodes priority fields.

the two specific properties in Section II-C, functions are categorized into two groups. Figure 4 visually distinguishes these groups, using yellow for the first and blue for the second. The first group includes functions that adjust the base scheduler node's priority fields and initiate updates, ensuring the job with the earliest absolute deadline receives the highest priority (Property 1), as detailed in Section VI-C. The second group is dedicated to applying updates and actual scheduling operations to enable preemptive EDF scheduling (Property 2), as discussed in Section VI-D.

Property 1: When a job is released or canceled, it is necessary to modify the priority fields in its base scheduler node. This ensures that the priority aligns with the current absolute deadline. This adjustment follows two primary pathways: 1) the PR Manager activates EDF scheduler functions, i.e., `_Scheduler_EDF_Release_job` and `_Scheduler_EDF_Cancel_job`; 2) `_Thread` functions are invoked to update internal priority fields, including:

- The preservation of the deadline-based priority value in the priority node.
- A new minimum priority upon insertion or modification in the red-black tree, requiring minimum priority updates in both the priority aggregation and the base priority node.
- The need to enqueue the task into the update queue if a new minimum priority is attributed to the task, facilitating subsequent updates to the EDF scheduler node.

Property 2: To ensure preemptive scheduling, the ready queue must be updated whenever a job's priority changes. The function `_Scheduler_EDF_Update_Priority` is called when tasks are in the update queue, leading to modifications in the priority field of the specialized EDF scheduler node. This is crucial for determining a task's position in the internal ready queue. Additionally, potential task preemption and changes in processor ownership need to be considered. These updates to the EDF scheduler node priority are reflected in the functions `_Scheduler_EDF_Schedule_body` and `_Scheduler_EDF_Update_heir`.

C. Verification for Deadline Driven Task Prioritization

This ~~section focuses on these functions that implement~~ subsection focuses on the functions for Property 1. Figure 5 provides an overview of the ~~modifications process.Only a selected number of critical functions are included for clarity and conciseness.~~ modification process.

~~Each~~ In RTEMS, each thread (or job) contains a priority aggregation tree that represents all the priority entries and extracts the highest priority of the current job. The job release or cancellation process involves one of the following three `_Thread` functions:

- `_Thread_Priority_changed` is used when the priority node is already part of the internal priority aggregation tree, necessitating an update to reflect the changes.
- `_Thread_Priority_insert` handles the addition of a new priority to the aggregation tree.
- `_Thread_Priority_remove` is responsible for removing an existing priority from the tree.

Afterwards, the `_Thread_Priority_apply` function is called with an associated priority action type. Each action type influences the specific tree operation during the red-black tree's internal update. It encapsulates both the priority node and the priority action type into a single object. This object is then utilized by the `_Thread_Priority_do_perform_actions` function, resulting in one of two possible outcomes:

- If the tree operation introduces a new minimum priority, it influences the task's positioning within the ready queue. Consequently, the scheduler node must be updated to reflect this new minimum value.
- If the tree operation does not result in a new minimum priority, no further action is required.

After inserting the priority into the tree (if exists), the `_Thread_queue_Context_add_priority_update` function is called if a new minimum priority value emerges. This purpose is to enqueue the task into a scheduler queue for subsequent updating.

1) Tasks Enqueueing and Tree Manipulations: This subsection summarizes the verification of task enqueueing and tree manipulations, focusing on the key behavior of the `_Priority_Non_empty_insert` function as a representative example. This function is responsible for inserting a new priority node into the scheduler's red-black tree when the corresponding deadline-based priority is not already present.

The verification ensures two main behaviors:

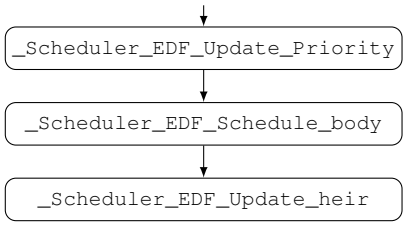

Fig. 6. Control flow of executing priority update

- When the newly inserted priority becomes the minimum in the tree, the scheduler updates the base scheduler node's priority and marks the associated task for re-queuing to reflect the new priority.
- If the insertion does not result in a new minimum, no priority update is needed, and the task remains unchanged in the queue.

To handle legacy implementation details that are difficult to model directly, abstract ghost variables are introduced to capture internal state transitions relevant to verification. The verification also ensures that interactions via function pointers, such as priority update callbacks, are explicitly tracked within the verification framework to maintain correctness guarantees.

2) Verification Challenges: During the verification of the `_Thread_Priority_do_perform_actions` function, a key challenge arose in validating the ghost pointer linked to the scheduler node. Specifically, Frama-C was unable to confirm the validity of the `queue_context` pointer after the `_Priority_Non_empty_insert` call, resulting in unmet preconditions for subsequent functions like `_Thread_queue_Context_add_priority_update`.

To resolve this, the tree helper functions, i.e., `_Priority_Extract_non_empty`, `_Priority_Changed`, and `_Priority_Non_empty_insert`, were inlined using Frama-C's `-inline-calls` option. This allowed Frama-C to validate `queue_context` directly within the target function, satisfying the necessary preconditions for task enqueueing.

A similar issue occurred with the `_Thread_Priority_apply` function. Although additional `\separated` preconditions improved pointer separation guarantees, full verification was only achieved by applying the same inlining strategy, ensuring pointer validity across function calls.

D. Verification for Preemptive EDF Scheduling Operations

In this ~~section~~subsection, we focus on the scheduler functions that realize theProperty 2. Following the above priority manipulation, the PR Manager calls the `_Scheduler_EDF_Update_Priority` function. This function is responsible for updating the priority field of the specialized EDF scheduler node and adjusting the ready queue accordingly. Since this update yields a task with an earlier absolute deadline than the currently executing task, preparations for a potential context switch are initiated. This involves updating the processor heir, i.e., the pointer to the next processor owner scheduled for execution. This procedure is illustrated in Figure 6. ~~Please note that the verification of these functions, which involve only minor variations in tree operations, is not reiterated here.~~

The function contracts ~~in this section~~ aim to verify specific postconditions, namely, ensuring the accurate reflection of the priority value in the specialized EDF scheduler node and the correct setting of the new processor heir under certain conditions. To ensure a structured and coherent analysis, the same bottom-up approach is applied, i.e., starting at the end of the function call chain.

1) Establishing a New Processor Heir: This verification targets the function, i.e., `Scheduler_Update_heir`, responsible for updating the processor's current task when a switch is required. A change is allowed only if the current task is preemptable and differs from the new task, with exceptions for forced dispatches in special cases.

The function updates the internal state to record the new processor heir and signals that a context switch is needed but does not perform the switch itself. Verification ensures that:

- When a processor switch is permitted, the internal state is updated correctly and a dispatch is signaled.
- When a switch is not allowed (e.g., the current task is non-preemptable), the system state remains unchanged.

This confirms that processor switches adhere to preemption rules and correctly trigger deferred dispatching.

2) Task Scheduling: This verification focuses on the function, i.e., `_Scheduler_EDF_Schedule_body`, responsible for promoting the highest-priority task from the ready queue to become the processor heir. The function updates the internal state to reflect the new heir and signals the need for a context switch when a change is required; otherwise, it leaves the system state unchanged.

~~To avoid low-level complexities related to the red-black tree implementation, the~~ The extraction of the highest-priority task is abstracted in the verification using ghost pointers. This ensures that ~~, despite underlying implementation details,~~ the verification confirms two key properties:

- When a new heir is selected, the internal scheduler state is updated correctly, and a dispatch is signaled,
- When no change is needed, the function preserves the current system state.

This guarantees that task selection and processor heir assignment follow EDF semantics reliably.

3) Priority Update: This verification covers the function, i.e., `_Scheduler_Update_priority` responsible for updating the priority of a task's scheduler node within the EDF framework. The function determines whether a priority update is needed and ensures that the EDF scheduler node is correctly synchronized with the base scheduler node's priority when required. The verification confirms that:

- If the task is not in a ready state, the function exits without modifying the system state,
- If the priorities are already aligned, no update occurs,

TABLE I
DERIVED PROOF GOALS AND REQUIRED TIME OF TWO PROPERTIES.

	Proof Goals		Time
	Qed	Alt-Ergo	
Deadline driven task prioritization	1110	262	$\approx 138s$
Preemptive EDF scheduling	167	8	$\approx 98s$

- When an update is necessary, the EDF node's priority is updated, and, if applicable, a new processor heir is selected and scheduled accordingly.

To handle the typecasting between scheduler node types, the verification leverages the memory model's support for pointer casts, ensuring that updates to the scheduler node maintain consistency and correctness within the EDF scheduling logic.

VII. RESULTS AND DISCUSSION

This section presents the verification results for the EDF scheduler properties and discusses key limitations arising from tool constraints and legacy aspects of the RTEMS codebase.

A. Verification Results

The EDF scheduler's implementation in RTEMS is distributed across various components, necessitating a holistic verification approach. Our achievements are realized through a combination of function contracts, as outlined in Section VI, with the underlying assumption that conversions from absolute deadlines to priority values and tree-related operations are correctly implemented and executed.

- Property 1: Our verification in Section VI-C confirms the correctness of task selection and its consistent identification of the minimal priority, corresponding to the earliest absolute deadline. Additionally, the functionality of the `_Scheduler_EDF_Map_priority` function is verified to ensure consistent prioritization between deadline-driven tasks and background tasks. Furthermore, the task first inserted into the red-black tree will be removed first when multiple tasks share the same priority, thereby maintaining consistent operation under conditions of identical task priorities.
- Property 2: The `_Scheduler_Update_heir` function encapsulates the preemption behavior, redefining the processor heir under specific conditions. Detailed discussions on preemption initiation and execution are available in Sections VI-D.

The derived proof goals for both build-in Qed and selected proofs to Alt-Ergo, and required time of the verification of two properties, i.e., deadline driven priority manipulation and preemptive EDF scheduling, via Frama-C and `wp` are presented in Table I. The process was executed with a single thread on an Intel Core i7-1360P with 32 GB main memory. For each function contract, it ends up at the second scale. ~~Comparing to [33], our study addresses unverified assign clauses. By validating the clauses, we enable the reuse of proofs in contracts of currently unverified functions. Our study focuses on functions that are crucial to scheduler operations, underlying locking protocols and priority modifications in RTEMS 5, which is orthogonal to the previous work.~~

B. Technical Limitations

1) Inline Functions and Verification Modularity: Certain contracts, such as the `Thread_Priority_apply` contract require an inline solution. While this may not cause significant concerns when considering the standalone function, problems arise when other functions call it. We were compelled to instruct Frama-C to treat those function calls as inline functions. However, the implications of this inline workaround on the overall verification process warrant further scrutiny. The inconsistency between treating a function as inline versus a standard function call, specifically in Frama-C, demands further exploration.

2) Limitations in Tree Functions and Memory Representation: The decision to bypass certain tree functions represents a potential limitation in accurately capturing the behavior and essence of these operations. Using ghost pointers as a transient abstraction mechanism might fail to reflect the true operations of the tree. This challenge is particularly evident in the scheduler's ready queue, where assumptions about the correct triggering of tree operations must be made due to the lack of direct access to the tree. As a result, the contents of the tree can only be hypothesized.

~~Certain tree-related properties, such as state adjustments, element placement according to key, and element extraction for the highest-priority task, remain unverified in Frama-C. This work focuses on the verification of the uni-processor EDF scheduler, considering only sequential accesses to tree-related data structures. However, in the case of global EDF scheduling, concurrent accesses to the same data structures introduce significant challenges and necessitate further research.~~

~~To verify these tree-related attributes more comprehensively, it is necessary to replace ghost pointer abstractions with genuine system objects and find a robust method to model the macros used by the tree library in the contracts.~~ Furthermore, the prevalent use of unsafe typecasts in the EDF implementation, as highlighted in Section ~~VI~~IV-B, could lead to potential mismatches. Currently, the memory model overlooks any potential issues arising from such typecasts.

C. Adaptability and Extendability

~~We formally define two essential properties of the EDF scheduler that all implementations in any RTOS must satisfy. This work applies a modular verification approach, demonstrating both the adaptability and extendability of our method.~~

Our ~~approach is designed to adapt to future RTEMS updates due to its modular structure. Currently, verification is based on~~ framework is resilient to updates in RTEMS, as verification

is modular and function-level. While the case study targets RTEMS version 5, ~~confirming the correctness of the EDF implementation under certain assumptions. If future updates retain the same functional primitives, our verification will remain valid with minimal adjustments, requiring only minor contract adaptations to accommodate any changes.~~

~~Our verification approach is~~ future changes to its EDF scheduler can be accommodated with minimal contract updates, provided core primitives remain intact. The framework is also architecture-independent~~as it focuses on the source code rather than the executable binary and does not include the BSP. Verifying the source code ensures the scheduler's correctness across all supported architectures, assuming the BSP is correctly implemented. The deployed architecture must fulfill all assumptions regarding the correctness of the underlying layers and primitives~~, operating at the source-code level without dependence on the binary or BSPs. Thus, as long as the underlying RTOS layers and BSP meet the required assumptions, correctness carries over to supported architectures.

~~The modular verification approach proposed here is extensible to other parts of RTEMS, such as task release control, and to EDF implementations in other RTOSes~~While our case study focuses on periodic task systems, the framework is applicable to sporadic and aperiodic workloads. Since the EDF scheduler does not include a job release manager, it remains agnostic to release semantics. As long as deadlines are correctly assigned, the scheduler preserves EDF behavior through its deadline-driven priority management.

Extending the framework to other RTOSes (e.g., FreeRTOS, Zephyr) is feasible if the EDF implementation follows a similar structural decomposition and the source code is compatible with formal verification tools. In this work, we use Frama-C and ACSL, but the framework may also be adapted to other tools such as VCC [11] or Dafny [29] with AutoCorres [20], assuming appropriate preprocessing and annotation capabilities. However, ~~formally defining properties relevant to the verification target, identifying functionsand primitives to verify, and determining the required assumptions for underlying layers and primitives requires a thorough understanding of the implemented algorithms~~generalization to other systems requires careful identification of relevant properties, affected functions, and verification boundaries, which needs further ~~research~~study.

VIII. Related Work

Providing a verified RTOS can be approached in two common ways. The first approach, developing a new RTOS with formal verification from proven formal properties, has several notable works. For an entire RTOS, earlier efforts include verifying the seL4 microkernel using HOL/Isabelle [26], which involved manually reimplementing parts of the microkernel. Gu et al. [21] presented CertiKOS, an architecture for concurrent operating system kernels, where each system layer is verified with Coq [6]. Guo et al. utilized the Proven Schedulability Analysis (PROSA) framework to link verified schedulability analyses with CertiKOS [22]. For the scheduler in an RTOS, Guo et al. [23] proposed implementing the scheduler in Gallina (a shallow embedding of a subset of C) and then verifying the code using Coq. The Gallina code is translated into C using a tool called Digger. Vanhems et al. [35] proposed a formal proof methodology for the EDF scheduler. They first proved the correctness of the election function algorithm using Coq and then elevated this proof to the implementation level through refinements based on a formally proven protokernel, available at https://pip.univ-lille.fr/, as cited in [25].

For the second approach, which focuses on the verification of existing systems, there are two main strategies. *Model Checking*-based approaches first require the module to be modeled in a formal language, with all necessary properties specified as logic formulas. A model checker is then employed to automatically verify these property specifications. As shown in [18], RTEMS code is modeled in Java, and a model checking procedure is performed using the Java Pathfinder. However, state explosion is a significant issue with model checking [5], [18]. Additionally, converting source code could introduce potential errors in the verification process, as it may not accurately represent the original code [5].

To address these challenges, *Deductive Verification* [13], [24] can be applied, allowing for direct verification of a program's properties based on its implementation. Several tools, such as VCC [11] (discontinued in 2016) and Frama-C (Framework for Modular Analysis of C programs) [12], support this approach. VCC was used to verify the Microsoft Hyper-V Hypervisor [28], and Frama-C was used to verify parts of the Paparazzi UAV autopilot, an open-source autopilot for drones [32]. Recently, Shi et al. [33] have introduced a framework based on deductive verification using Frama-C for formally verifying resource synchronization protocols in an RTOS. Their framework assumes all underlying layers are correctly implemented~~, a condition that might not always hold true in real-world situations.~~ . Besides these approaches, Nicole et al. [31] developed a fully automated verification process for small OS kernels, focusing on their binary executables. This process ensures the absence of privilege escalation (APE) and runtime errors (ARTE). However, it does not verify whether the implementation aligns with the original design by checking all these formally described properties.

Additionally, other formal verification approaches only focus on the proof of timing analysis or schedulability analysis for existing strategies, without considering the real implementations. For example, Bozhko et al. [8] established a formal foundation for the busy-window principle through an abstract response-time analysis that is independent of specific scheduling policies and workload models. All definitions and proofs in their work have been mechanized and verified with the Coq proof assistant.

IX. Conclusion

Many RTOSes, including those developed by the open-source community, are increasingly deployed in safety-critical domains, underscoring the need for rigorous verification of

their core components. In this work, we proposed a general framework for verifying the correctness of EDF scheduler implementations in RTOSes. We formally defined two fundamental correctness properties that all EDF schedulers must satisfy, outlined the typical software structure underlying EDF implementations, and introduced a modular deductive verification methodology. To demonstrate its applicability, we applied the framework to the uniprocessor EDF scheduler in RTEMS, performing the first deductive verification of its implementation under clearly defined assumptions, including the correctness of supporting kernel primitives and ready queue operations. Our results show that such a modular verification framework can effectively ensure ~~EDF correctness~~the correct implementation of EDF using multiple OS primitives, even in legacy or repurposed RTOS infrastructures. This work contributes toward improving the reliability and assurance of real-time systems and provides a foundation for future research in verified RTOS components.

References

[1] An SMT Solver For Software Verification. https://alt-ergo.ocamlpro.com/, 2014. Last accessed on 2025-05-08.

[2] Alexander Pinheiro Paschoaletto. Potential Issues with the EDF Scheduler in Zephyr, 2024. https://github.com/zephyrproject-rtos/zephyr/issues/79361, Last accessed on 2025-05-18.

[3] Amazon. FreeRTOS: Real-time operating system for microcontrollers, 2017. https://www.freertos.org/, Last accessed on 2025-05-18.

[4] Anonymous Authors. Deployment Instructions and Function Contracts for Formally Verifying Implementations of Earliest Deadline First Scheduler in RTEMS, 2025. https://anonymous.4open.science/r/Formally-Verifying-Implementations-of-EDF-Scheduler-in-RTEMS-0F82/, Last accessed on 2025-05-18.

[5] C. Baier and J.-P. Katoen. *Principles of model checking*. MIT press, 2008.

[6] Y. Bertot and P. Castéran. *Interactive Theorem Proving and Program Development - Coq'Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2004.

[7] F. Bobot, J.-C. Filliâtre, C. Marché, and A. Paskevich. Why3: Shepherd your herd of provers. *Boogie 2011: First International Workshop on Intermediate Verification Languages*, 2011.

[8] S. Bozhko and B. B. Brandenburg. Abstract response-time analysis: A formal foundation for the busy-window principle. In M. Völp, editor, *32nd Euromicro Conference on Real-Time Systems, ECRTS*, LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.

[9] G. C. Buttazzo. Rate monotonic vs. EDF: judgment day. *Real Time Syst.*, 29(1):5–26, 2005.

[10] K.-H. Chen, G. von Der Brüggen, and J.-J. Chen. Overrun Handling for Mixed-Criticality Support in RTEMS. In *Workshop on Mixed Criticality Systems*, 2016.

[11] E. Cohen, M. Dahlweid, M. A. Hillebrand, D. Leinenbach, M. Moskal, T. Santen, W. Schulte, and S. Tobies. VCC: A practical system for verifying concurrent C. In *Theorem Proving in Higher Order Logics, 22nd International Conference, Proceedings*. Springer, 2009.

[12] P. Cuoq, F. Kirchner, N. Kosmatov, V. Prevosto, J. Signoles, and B. Yakobowski. Frama-c: A software analysis perspective. In *Software Engineering and Formal Methods SEFM*. Springer, 2012.

[13] R. W. Floyd. Assigning meanings to programs. *Proceedings of Symposium on Applied Mathematics*, 19:19–32, 1967.

[14] L. Foundation. Zephyr project. https://zephyrproject.org/, 2016. Last accessed on 2025-05-08.

[15] FRAMA-C. Frama-C Software Analyzers. https://frama-c.com/, Last accessed on 2025-04-28.

[16] FRAMA-C. ANSI/ISO C Specification Language, 2007. https://frama-c.com/html/acsl.html, Last accessed on 2025-04-28.

[17] FRAMA-C. WP Plugin, 2007. https://frama-c.com/fc-plugins/wp.html, Last accessed on 2025-04-28.

[18] S. Gadia, C. Artho, and G. Bloom. Verifying nested lock priority inheritance in RTEMS with java pathfinder. In *18th International Conference on Formal Engineering Methods, Proceedings*, 2016.

[19] C. Garion, G. Hattenberger, B. Pollien, P. Roux, and X. Thirioux. A gentle introduction to c code verification using the frama-c platform, 2022.

[20] D. Greenaway. *Automated proof-producing abstraction of C code*. PhD thesis, University of New South Wales, Sydney, Australia, 2014.

[21] R. Gu, Z. Shao, H. Chen, X. N. Wu, J. Kim, V. Sjöberg, and D. Costanzo. Certikos: An extensible architecture for building certified concurrent OS kernels. In *12th OSDI*, 2016.

[22] X. Guo, M. Lesourd, M. Liu, L. Rieg, and Z. Shao. Integrating formal schedulability analysis into a verified OS kernel. In *Computer Aided Verification - 31st International Conference, CAV*. Springer, 2019.

[23] X. Guo, L. Rieg, and P. Torrini. A generic approach for the certified schedulability analysis of software systems. In *Embedded and Real-Time Computing Systems and Applications (RTCSA)*. IEEE, 2021.

[24] C. A. R. Hoare. An axiomatic basis for computer programming. *Commun. ACM*, 12(10):576–580, 1969.

[25] N. Jomaa, P. Torrini, D. Nowak, G. Grimaud, and S. Hym. Proof-oriented design of a separation kernel with minimal trusted computing base. *Electron. Commun. Eur. Assoc. Softw. Sci. Technol.*, 76, 2018.

[26] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, et al. sel4: Formal verification of an os kernel. In *Operating systems principles*, 2009.

[27] P. Lammich and T. Nipkow. Priority search trees. *Arch. Formal Proofs*, 2019.

[28] D. Leinenbach and T. Santen. Verifying the microsoft hyper-v hypervisor with vcc. In *FM 2009: Formal Methods*, pages 806–809, 2009.

[29] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. In E. M. Clarke and A. Voronkov, editors, *Logic for Programming, Artificial Intelligence, and Reasoning - 16th International Conference, LPAR*, volume 6355 of *Lecture Notes in Computer Science*, pages 348–370. Springer, 2010.

[30] H. Liang, D. Zhang, X. Pei, X. Jia, G. Li, and J. Xu. A correctness verification method for c programs based on vcc. In *International Conference on Cyber Security and Cloud Computing*. IEEE, 2016.

[31] O. Nicole, M. Lemerre, S. Bardin, and X. Rival. No crash, no exploit: Automated verification of embedded kernels. In *27th IEEE Real-Time and Embedded Technology and Applications Symposium*, 2021.

[32] B. Pollien, C. Garion, G. Hattenberger, P. Roux, and X. Thirioux. Verifying the mathematical library of an uav autopilot with frama-c. In *26th International Conference FMICS*. Springer, 2021.

[33] J. Shi, C. von Egidy, K. Chen, and J. Chen. Formal verification of resource synchronization protocol implementations: A case study in RTEMS. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, 2022.

[34] The RTEMS Project. Real-Time Executive for Multiprocessor Systems (RTEMS), 2014. http://www.rtems.org/, Last accessed on 2025-04-28.

[35] F. Vanhems, V. Rusu, D. Nowak, and G. Grimaud. A formal correctness proof for an EDF scheduler implementation. In *28th IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS*, 2022.